

code cannot.

Summary Comments

Explaining the programmer's intent isn't the only way comments can be useful. Brief comments that summarize several lines of code allow the reader to skim the source code and get a general idea of what it does. Programmers often use a blank space to separate "paragraphs" of code from each other, and the summarizing comments usually occupy one line at the start of these paragraphs. Unlike one-line comments that explain single lines of code, the summarizing comments describe what the code does at a higher level of abstraction.

For example, you can tell from reading these four lines of code that they set the `playerTurn` variable to a value representing the opposite player. But the short, single-line comment spares the reader from having to read and reason about the code to understand the purpose of doing this:

```
# Switch turns to other player:
if playerTurn == PLAYER_X:
    playerTurn = PLAYER_O
elif playerTurn == PLAYER_O:
    playerTurn = PLAYER_X
```

A scattering of these summary comments throughout your program makes it much easier to skim. The programmer can then take a closer look at any particular points of interest. Summary comments can also prevent programmers from developing a misleading idea about what the code does. A brief, summarizing comment can confirm that the developer properly understood how the code works.

"Lessons Learned" Comments

When I worked at a software company, I was once asked to adapt a graph library so it could handle the real-time updates of millions of data points in a chart. The library we were using could either update graphs in real time *or* support graphs with millions of data points, but not both. I thought I could finish the task in a few days. By the third week, I was still convinced that I